

PRISMFLOW - MASTER PROJECT CREATION GUIDE

A complete B.Tech 2nd-year friendly project roadmap, architecture, pipeline, file-generation plan, research experiments, and Claude prompts.

CORE IDEA

PrismFlow is a Multi-View Learning system. It receives multiple views of the same sample, extracts information from each view, checks how independent those views really are, estimates an effective number of independent views (ENIV), discounts duplicated or suspicious evidence, fuses the remaining evidence, and outputs a prediction together with confidence and diagnostics.

CENTRAL RESEARCH QUESTION

More agreeing views do not automatically mean more independent information. PrismFlow investigates whether a system can recognize redundant or coordinated agreement and avoid becoming overconfident because several views repeat the same information or are deliberately manipulated together.

ONE-SENTENCE THESIS

Agreement between views should contribute evidence according to how independent the views really are, not merely according to how many views agree.

IMPORTANT RESEARCH DISCIPLINE

Do not build everything at once. Build one hypothesis and one experiment at a time. Most neural components are standard. The research contribution is mainly in the measurement, dependence estimation, ENIV, attack model, calibration, and evaluation protocol.

1. HIGH-LEVEL SYSTEM FLOW

INPUT MULTI-VIEW DATA

|

v

DATA VALIDATION / PREPROCESSING

|

v

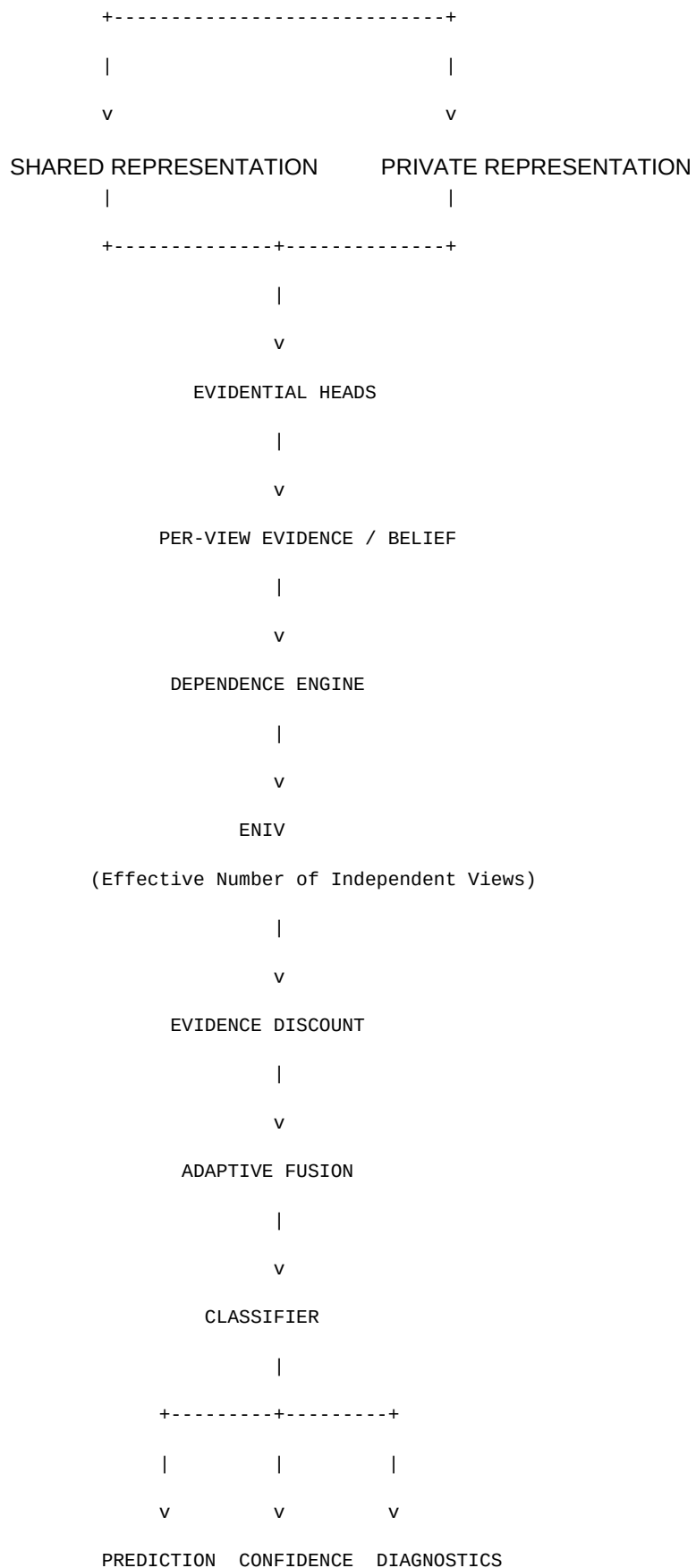
VIEW-SPECIFIC ENCODERS

|

v

FEATURE REPRESENTATIONS

|



Diagnostics may include:

- per-view evidence
- per-view uncertainty
- dependence matrix
- ENIV
- missing/noisy view status
- suspicious agreement warning
- calibration metrics during evaluation

2. COMPONENTS - IN EASY LANGUAGE

1) Dataset / Input Manager

Receives all views belonging to one sample and keeps them aligned.

2) Preprocessing

Cleans, normalizes, resizes, masks missing views, and simulates controlled noise/corruption.

3) View Encoders

Each view is converted into a feature vector. Use view-specific encoders; do not force all views through the same weights by default.

4) Shared / Private Representation

Separate information common to multiple views from information unique to one view.

5) Evidential Head

Produces evidence, belief/probabilities, and uncertainty for each view.

6) Dependence Engine

Measures whether two views contain strongly related or repeated information.

7) ENIV

Estimates how many independent views you effectively have. Six highly redundant views may behave more like two or three independent sources.

8) Evidence Discount

Reduces the influence of redundant/low-independence evidence instead of letting repeated agreement create fake confidence.

9) Fusion

Combines the adjusted evidence from all views.

10) Classifier

Produces the final prediction.

11) Chorus Attack Simulator

In a controlled research experiment, several views are manipulated toward the same wrong class to test whether ordinary fusion becomes overconfident.

12) Calibration / Evaluation

Checks not only whether predictions are correct, but whether confidence is honest.

13) GUI

Desktop interface for the final demo.

14) EXE Packaging

Packages the application so the user can launch PrismFlow.exe directly on Windows.

3. V1 VS ADVANCED VERSIONS

V1 - Build this first:

- view-specific encoders
- evidential head
- simple dependence estimator
- design-effect ENIV
- evidence discount
- fusion
- classifier
- baseline experiments
- clone experiment
- missing/noisy view experiments
- basic calibration

V2 / Advanced:

- per-sample ENIV
- learned dependence estimator
- Shapley-based ENIV
- tail dependence
- adaptive adversarial evaluation
- out-of-band trust signals
- SOC application
- independence-budget dashboard

Do NOT start with flow matching, PINNs, PID, MINE/InfoNCE, or other advanced research terms. Add them only after the V1 hypothesis is measured.

4. RECOMMENDED REPOSITORY STRUCTURE

```
PrismFlow/  
|  
+- data/  
| +- raw/  
| +- processed/  
| +- splits/  
|  
+- models/  
| +- encoders/  
| +- evidence/  
| +- fusion/  
| +- classifier/  
| +- prismflow/  
|  
+- prismflow/  
| +- data/  
| +- preprocessing/  
| +- models/  
| +- statistics/  
| +- env/  
| +- attacks/  
| +- evaluation/  
| +- calibration/  
| +- utils/  
| +- gui/  
|  
+- experiments/  
| +- baseline/  
| +- clone/
```

```
| +- missing/
| +- noise/
| +- chorus/
| +- per_sample_eniv/
|
+- tests/
| +- unit/
| +- integration/
|
+- configs/
+- checkpoints/
+- results/
+- logs/
+- docs/
+- train.py
+- evaluate.py
+- run.py
+- run_app.py
+- requirements.txt
+- pyproject.toml
+- README.md
```

5. COMPLETE DEVELOPMENT PIPELINE

P00 - Architecture contract

P01 - Project skeleton

P02 - Configuration + utilities

P03 - Dataset/input pipeline

P04 - Preprocessing

P05 - View encoders

P06 - Evidential head

P07 - Shared/private representation

P08 - Dependence engine

P09 - ENIV V1

P10 - Evidence discount

P11 - Fusion + classifier

P12 - Baseline experiment
P13 - Clone experiment
P14 - Missing/noisy experiments
P15 - Chorus attack
P16 - PrismFlow defence
P17 - Calibration/research evaluation
P18 - Per-sample ENIV
P19 - Desktop GUI + EXE
P20 - Final integration + audit

Recommended implementation sequence:

P00 -> P01 -> P02 -> P03 -> P04 -> P05 -> P06 -> P08 -> P09 -> P10 -> P11 -> P12 -> P13 -> P14 -> P15 ->
P16 -> P17 -> P18 -> P19 -> P20

P07 is useful but can remain minimal in early V1.

6. RESEARCH HYPOTHESES

H1: Duplicating a view can artificially increase confidence in naive fusion.
H2: Multiple coordinated wrong views can cause ordinary fusion to become overconfident.
H3: Dependence-aware ENIV can identify redundant evidence.
H4: Dependence discounting can reduce confidence inflation.
H5: PrismFlow can maintain better calibration under missing/noisy/collusive-view conditions.
H6: Per-sample ENIV can detect situations where views become dependent only for particular inputs.

Every hypothesis must be tested with real experiment results. Never invent numbers.

7. FINAL OUTPUT OF PRISMFLOW

Example output:

PRISMFLOW RESULT

Prediction: MALICIOUS

Confidence: 82.4%

Uncertainty: 0.17

Nominal views: 5

Effective independent views (ENIV): 2.8

Per-view status:

View 1 - Healthy - evidence 0.91

View 2 - Healthy - evidence 0.84

View 3 - Suspicious - evidence 0.31

View 4 - Healthy - evidence 0.79

View 5 - Related to View 2 - discounted

Warning:

Suspicious correlated agreement detected.

The application should show the prediction plus the reasoning/diagnostics, not just a class label.

8. 21 CLAUDE GENERATION PARTS

The prompts below are designed to be run sequentially. Each one tells Claude to inspect the existing repository, modify only the relevant scope, run tests, and stop. This limits context growth and prevents project-wide regeneration.

PART 00 - ARCHITECTURE CONTRACT

You are the lead software architect for the PrismFlow research project.

PROJECT NAME:

PrismFlow - Dependence-Discounted Evidential Fusion under Collusive View Compromise.

PROJECT GOAL:

Build a research-grade Multi-View Learning system that investigates whether multiple agreeing views are actually independent sources of evidence.

CORE HYPOTHESIS:

Agreement between views should contribute evidence according to their effective independence, not merely according to the number of views.

IMPORTANT:

This is a staged project. Do NOT generate the complete project now.

Your task in this part is ONLY to create the architecture contract and project engineering rules.

Define:

1. System layers: data, preprocessing, encoders, evidential, shared/private, dependence, ENIV, discount, fusion, prediction, attack simulation, evaluation, GUI.
2. Full data flow: raw views -> preprocessing -> features -> evidence -> dependence -> ENIV -> discounting -> fusion -> prediction -> confidence -> diagnostics.
3. Interfaces between major modules.
4. For every component classify it as trainable neural module, statistical estimator, closed-form operation, experiment, or engineering component.
5. Do not turn ENIV or collusion detection into arbitrary neural layers.
6. Define V1 versus future versions. V1: view-specific encoders, evidential head, simple dependence estimator, design-effect ENIV, evidence discounting, fusion, classifier, baseline evaluation.
7. Repository naming conventions.
8. Tensor shape conventions.
9. Error handling.
10. Reproducibility rules: seeds, configs, experiment IDs, checkpoints, saved metrics.
11. Rules for later Claude sessions: inspect first; never rebuild; preserve APIs; do not silently rename; no unneeded dependencies; separate V1 from advanced experiments; add tests.

OUTPUT ONLY:

docs/ARCHITECTURE.md
docs/DEVELOPMENT_RULES.md
docs/DATA_FLOW.md
docs/RESEARCH_SCOPE.md

Do not generate implementation code. Stop.

PART 01 - PROJECT SKELETON

Read all architecture documents and inspect the existing repository.
Create ONLY the PrismFlow project skeleton.

Create directories and minimal `__init__.py` files for:

prismflow/
prismflow/data/
prismflow/preprocessing/
prismflow/models/
prismflow/models/encoders/
prismflow/models/evidence/
prismflow/models/fusion/
prismflow/models/classifier/
prismflow/statistics/
prismflow/eniv/
prismflow/attacks/
prismflow/evaluation/
prismflow/calibration/
prismflow/utils/
prismflow/gui/

data/raw data/processed data/splits experiments/baseline experiments/clone experiments/missing experiments/noise experiments/chorus tests/unit tests/integration checkpoints results logs docs

Create requirements.txt, README.md, .gitignore, and pyproject.toml if useful.

Do not implement ML.

Validate import/package structure and stop.

PART 02 - CONFIGURATION AND UTILITIES

Inspect the repository first.

Implement ONLY shared engineering infrastructure.

Create:

prismflow/utils/config.py
prismflow/utils/seed.py
prismflow/utils/logging.py
prismflow/utils/device.py
prismflow/utils/checkpoint.py
prismflow/utils/serialization.py
configs/default.yaml

Requirements: centralized configuration, no hard-coded paths, reproducible seeds, CPU/GPU detection, consistent logging, checkpoint save/load, JSON/YAML serialization, type hints, error handling, tests.

Do not build encoders, fusion, ENIV, attacks, or GUI.

Run tests, verify config, checkpoint and seed reproducibility, stop.

PART 03 - MULTI-VIEW DATASET PIPELINE

Implement the multi-view dataset layer.

Create:

prismflow/data/dataset.py

prismflow/data/sample.py

prismflow/data/loaders.py

prismflow/data/splits.py

prismflow/data/validation.py

Support per sample:

sample_id, view_1...view_N, label, view availability mask, optional metadata.

Support complete, missing, noisy, duplicated and corrupted views.

Batch representation should include views, view_mask, labels, sample_ids, metadata.

Requirements: deterministic split, configurable view count, missing mask, same-sample validation, no label leakage, unit tests, synthetic toy dataset.

Run a toy batch and print tensor shapes. Do not implement neural networks. Stop.

PART 04 - PREPROCESSING

Implement only preprocessing.

Create:

prismflow/preprocessing/base.py

prismflow/preprocessing/normalization.py

prismflow/preprocessing/view_preprocessor.py

prismflow/preprocessing/corruption.py
prismflow/preprocessing/masking.py

Support per-view configuration, train-only fitting to prevent leakage, normalization, view-specific transformation, missing masks, controlled noise/corruption, reproducibility, sample/view alignment.

Add tests for consistency, missing views, corruption reproducibility and train/test isolation.

Do not implement encoders. Stop after tests pass.

PART 05 - VIEW ENCODERS

Inspect the repository and preserve interfaces.

Create:

prismflow/models/encoders/base_encoder.py
prismflow/models/encoders/mlp_encoder.py
prismflow/models/encoders/view_encoder.py
prismflow/models/encoders/encoder_factory.py

Requirements:

- each view can have its own encoder
- do not silently share weights
- common output dimension
- configurable input/hidden/feature dimensions and dropout
- missing-view handling through mask
- structured output with features and mask
- easy replacement later with CNN/ViT/Transformer
- tests for forward pass, dimensions, missing views, parameter independence, gradient flow.

Add a toy training script. Do not implement evidential learning or ENIV. Stop.

PART 06 - EVIDENTIAL HEAD

Implement evidential learning.

Create:

prismflow/models/evidence/edl_head.py
prismflow/models/evidence/dirichlet.py
prismflow/models/evidence/losses.py
prismflow/models/evidence/annealing.py

Implement:

feature -> evidence -> Dirichlet parameters -> belief -> uncertainty.
Use non-negative evidence and $\alpha = \text{evidence} + 1$.
Expose α , evidence, belief, uncertainty and class probabilities.

Implement a Type-II maximum-likelihood-style evidential loss and KL regularization with configurable early-epoch annealing. Do not start KL at full strength.

Tests: non-negative evidence, $\alpha > 0$, valid probabilities, sensible uncertainty, finite loss, correct annealing.
Do not implement ENIV or dependence estimation. Stop.

PART 07 - SHARED / PRIVATE REPRESENTATION

Inspect current interfaces.

Create:

prismflow/models/shared_private.py
prismflow/statistics/hsic.py
prismflow/models/disentanglement_losses.py

Implement shared and private feature branches.

Use HSIC as the first independence constraint.

Requirements: configurable dimensions, differentiable implementation, stable batch computation, tests, numerical-stability checks.

Compare with and without disentanglement on a toy experiment.

Do not implement ENIV. Stop.

PART 08 - DEPENDENCE ENGINE

Implement V1 dependence analysis.

Create:

prismflow/statistics/dependence.py
prismflow/statistics/correlation.py
prismflow/statistics/distance_correlation.py

Start with Pearson correlation of per-view evidence and distance correlation where practical.

Return a symmetric dependence matrix.

Support missing views, diagonal handling, bounded values and numerical stability.

Add tests and heatmap/export visualization.

Do not implement ENIV yet. Stop.

PART 09 - ENIV V1

Create:

prismflow/eniv/design_effect.py
prismflow/eniv/eniv.py
tests/unit/test_eniv.py

V1 formula:

$$n_{\text{eff}} = n / (1 + (n - 1) * \text{mean_pairwise_dependence})$$

Requirements:

- available view count
- dependence matrix
- ignore unavailable views
- return nominal count, effective count, efficiency ratio, mean dependence
- $1 \leq \text{effective count} \leq \text{available views}$
- handle one/zero views, independence, high correlation and missing views.

Do not make ENIV trainable.

Generate plots of nominal versus effective count.

Create docs/ENIV_V1.md.

Do not connect it to training yet. Stop.

PART 10 - EVIDENCE DISCOUNT

Create:

prismflow/eniv/discount.py

tests/unit/test_discount.py

Inputs:

per-view belief, uncertainty, effective view count, nominal available view count.

Compute $\alpha = n_{\text{eff}} / n$ and apply a dependence discount before fusion.

Preserve valid constraints, increase uncertainty when evidence is discounted, support missing views, and expose original vs discounted evidence.

Test $\alpha=1$, $\alpha<1$, one view, multiple views and numerical stability.

Do not modify encoders.

Do not backpropagate through ENIV in V1.

Stop.

PART 11 - FUSION + CLASSIFIER

Integrate:

encoders, evidential heads, dependence engine, ENIV and discount module.

Create:

prismflow/models/fusion/evidence_fusion.py

prismflow/models/fusion/fusion.py

prismflow/models/classifier/head.py

prismflow/models/prismflow_v1.py

Pipeline:

views -> encoders -> evidence -> dependence -> ENIV -> discount -> fusion -> classifier.

Return:

prediction, class probabilities, confidence, uncertainty, per-view evidence, per-view belief, per-view uncertainty, dependence matrix, ENIV, view mask.

Add end-to-end forward test and toy training script.
Do not implement attacks. Stop.

PART 12 - BASELINE EXPERIMENT

Create:

experiments/baseline/train_baseline.py
experiments/baseline/evaluate_baseline.py
experiments/baseline/config.yaml
experiments/baseline/README.md

Compare:

- 1 single-view model
- 2 simple feature concatenation
- 3 ordinary multi-view fusion
- 4 PrismFlow V1

Report accuracy, precision, recall, F1, confidence and uncertainty.
Save results under results/baseline/.
Do not optimize for impressive numbers; establish trustworthy baselines.
Verify on toy data. Stop.

PART 13 - CLONE / DUPLICATE VIEW EXPERIMENT

Create:

experiments/clone/clone_views.py
experiments/clone/run_clone_experiment.py
experiments/clone/plot_results.py
experiments/clone/README.md

Start with A,B,C,D. Then duplicate one view progressively.
Compare ordinary fusion vs PrismFlow V1.
Measure confidence, uncertainty, ENIV, accuracy and ECE where available.
Produce plots:
confidence vs duplicate count

ENIV vs duplicate count
uncertainty vs duplicate count

Main question:

Does duplicating an existing view increase confidence even though no new information has been added?

Do not invent results. Stop.

PART 14 - MISSING AND NOISY VIEWS

Create:

experiments/missing/run_missing.py

experiments/noise/run_noise.py

experiments/missing/README.md

experiments/noise/README.md

Test missing rates 10%, 30%, 50%, 70%.

Test one noisy view, multiple noisy views, and severely corrupted view.

Compare ordinary multi-view vs PrismFlow V1.

Measure accuracy, confidence, uncertainty, ECE and ENIV.

Generate plots.

Never claim superiority unless measurements show it. Stop.

PART 15 - CHORUS ATTACK

Implement a controlled research simulation against our own models only.

Create:

prismflow/attacks/chorus.py

prismflow/attacks/attack_utils.py

experiments/chorus/run_chorus.py

experiments/chorus/config.yaml

experiments/chorus/README.md

Select k compromised views. Optimize them toward the same incorrect target class and mutual agreement.

Support $k=1,2,3$ and configurable perturbation strength.
Compare ordinary fusion, PrismFlow without discount and PrismFlow with discount.

Measure attack success rate, wrong-class confidence, uncertainty, dependence, ENIV and calibration.
Do not implement real-world intrusion. Use synthetic/public benchmark data and controlled inputs only.
Save perturbations and metadata. Stop.

PART 16 - PRISMFLOW DEFENCE

Integrate the complete V1 defence.

Create:
prismflow/models/prismflow_defended.py

Pipeline:
input -> encoders -> evidential heads -> dependence -> ENIV -> dependence discount -> evidential fusion -> prediction.

Expose prediction, confidence, uncertainty, ENIV, dependence matrix, per-view reliability, per-view evidence and warning flags.

Suspicious-agreement warning should arise from model/statistical quantities, not hard-coded knowledge of the attack.
Compare baseline, ordinary fusion, PrismFlow without discount and PrismFlow with discount.
Run complete tests. Stop.

PART 17 - CALIBRATION + RESEARCH EVALUATION

Create:
prismflow/evaluation/metrics.py
prismflow/calibration/ece.py
prismflow/calibration/brier.py
prismflow/calibration/auroc.py

evaluate_all.py

a unified protocol for:

Accuracy

ECE

Brier score

Brier reliability/resolution/uncertainty decomposition where practical

AURC

selective risk

attack coverage

confidence under attack

Every experiment must produce the same standardized CSV/JSON/plot format.

Create docs/EVALUATION_PROTOCOL.md.

Do not modify model architecture. Stop.

PART 18 - PER-SAMPLE ENIV / ADVANCED RESEARCH

This is an advanced stage. Do not replace V1.

Create:

prismflow/eniv/per_sample.py

prismflow/eniv/amortized_estimator.py

experiments/per_sample_eniv/

Goal: estimate effective independence per input sample.

The estimator receives evidence from available views and estimates sample-conditional dependence.

Requirements:

- preserve V1 fallback
- configurable enable/disable
- no destructive changes
- no leakage
- document objective and limitations
- start lightweight
- do not immediately implement MINE, PID or highly complex estimators.

Compare global ENIV vs per-sample ENIV on stability, calibration, attack detection and compute.

If unstable, keep V1 as default. Stop.

PART 19 - DESKTOP GUI + EXE

Build a Windows desktop application using PySide6.
Do not rewrite the ML architecture.

Create:

prismflow/gui/main_window.py
prismflow/gui/input_panel.py
prismflow/gui/view_panel.py
prismflow/gui/dependence_panel.py
prismflow/gui/results_panel.py
prismflow/gui/plots.py
prismflow/gui/app_controller.py
run_app.py

Screens:

- 1 experiment/input selection
- 2 view status: available/missing/noisy
- 3 analysis: Run PrismFlow
- 4 results: prediction, confidence, uncertainty, ENIV, dependence matrix, per-view evidence, warnings
- 5 experiment visualization: confidence chart, ENIV chart, dependence heatmap

The GUI must call the existing engine, not duplicate model logic.

Add PyInstaller packaging configuration.

Target PrismFlow.exe.

Support CPU inference.

Test startup, model loading, inference, missing views and corrupted inputs before packaging.

Stop.

PART 20 - FINAL INTEGRATION + AUDIT

Perform a complete production-readiness and research-readiness audit.

Do not redesign unless required to fix a critical bug.

Inspect all modules, configs, dataset, model pipeline, ENIV, dependence, discount, fusion, attacks, metrics, experiments, GUI and packaging.

Verify:

input -> preprocessing -> view encoders -> evidence -> shared/private -> dependence -> ENIV -> discount -> fusion -> prediction -> confidence -> diagnostics -> GUI

Run:

- 1 unit tests
- 2 integration tests
- 3 toy end-to-end test
- 4 baseline
- 5 clone
- 6 missing view
- 7 noise
- 8 Chorus Attack
- 9 PrismFlow defence
- 10 GUI smoke test

Check for:

accidental weight sharing, data leakage, label leakage, hard-coded results, hidden attack labels, invalid probabilities, NaN/Inf, non-reproducibility, broken checkpoint loading, import cycles, duplicated logic, unused dependencies, inconsistent tensor shapes, misleading metrics.

Create:

docs/FINAL_ARCHITECTURE.md
docs/EXPERIMENT_MATRIX.md
docs/KNOWN_LIMITATIONS.md
docs/REPRODUCIBILITY.md
docs/INSTALLATION.md
docs/DEMO_GUIDE.md
results/final_summary.json
results/final_summary.csv
AUDIT_REPORT.md

Classify everything as WORKING, PARTIALLY WORKING, EXPERIMENTAL, NOT IMPLEMENTED, or KNOWN LIMITATION.

Never fabricate results.

If an experiment has not completed successfully, mark it NOT VERIFIED.

Stop.

9. GOLDEN RULES FOR EVERY CLAUDE SESSION

Before changing anything:

1. Inspect the repository.
2. Read docs/ARCHITECTURE.md.
3. Read docs/DEVELOPMENT_RULES.md.
4. Read the relevant previous module files.
5. Reuse existing interfaces.
6. Modify only files relevant to the current part.
7. Run tests.
8. Do not fabricate results.
9. Do not silently change public APIs or tensor shapes.
10. Do not add dependencies without necessity and documentation.
11. Never regenerate the entire repository unless explicitly asked.
12. Keep research experiments reproducible.

10. CORE PRINCIPLE TO REMEMBER

PrismFlow is not valuable because it contains the largest number of fancy AI techniques.

It is valuable if it can demonstrate, with controlled experiments, that:

- agreement can come from redundant information;
- coordinated agreement can fool ordinary fusion;
- effective independence can be estimated;
- confidence can be discounted when apparent agreement is not independent;
- calibration becomes more trustworthy under difficult multi-view conditions.

START SMALL.

PROVE EACH HYPOTHESIS.

THEN SCALE.